

Assignment-20 (Lab-20.2)

Course : AI Assisted Coding

Topic: Security Testing: Identifying Vulnerabilities in AI-Generated Code

Student Details

Name :Batti Chandan Singh

Hall Ticket :2303A51515

Batch :22

Task-01

Final Optimal Prompt

Generate a Python user registration and login system. First show insecure plain-text password storage, then refactor using secure hashing (hashlib). Include error handling for invalid inputs and ensure passwords are never printed.

Code Screenshot

```
6 import hashlib
7 users = {}
8
9 def hash_password(password):
10     return hashlib.sha256(password.encode()).hexdigest()
11
12 def register():
13     username = input("Enter username: ")
14     password = input("Enter password: ")
15
16     if not username or not password:
17         print("Error: Username and password cannot be empty.")
18         return
19
20     users[username] = hash_password(password)
21     print("User registered successfully.")
22
23 def login():
24     username = input("Enter username: ")
25     password = input("Enter password: ")
26
27     if username in users and users[username] == hash_password(password):
28         print("Login successful")
29     else:
30         print("Invalid credentials")
31
32 register()
33 login()
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assist
Enter username: chandu
Enter password: diehr8273$
User registered successfully.
Enter username: chandu
Enter password: diehr8273$
Login successful
PS D:\6th Sem 2025-26\AI-Assist
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Initially, storing passwords in plain text is highly insecure because attackers can directly read sensitive user data if the database is compromised. In this task, hashing is introduced using SHA-256, which converts the password into a fixed-length encrypted format. This ensures that even if the data is leaked, the original password cannot be easily retrieved. The system compares hashed values during login, improving security without exposing raw passwords. Error handling is added to prevent empty inputs, making the system more robust. This approach reflects responsible AI-assisted coding practices by prioritizing user data protection and secure authentication mechanisms.

Task-02

Final Optimal Prompt

Generate Python code that accesses an API using an API key. Avoid hardcoding the key and instead read it from environment variables. Include error handling if the key is missing or invalid.

Code Screenshot

```
Include error handling if the key is missing or invalid.
import os

def get_api_data():
    api_key = os.getenv("API_KEY")

    if not api_key:
        print("Error: API key not found in environment variables.")
        return

    valid_key = "12345-ABCDE"
    if api_key == valid_key:
        print("API request successful")
    else:
        print("Invalid API key")

if __name__ == "__main__":
    get_api_data()
```

Output Screenshot

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TASKS COMMENTS
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> python "d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-20(Lab-20.2).py"
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\python 3.13.6\python.exe" "d:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file\Assignment-20(Lab-20.2).py"
API request successful
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> 
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Hardcoding API keys directly in source code is a major security risk, as it exposes sensitive credentials if the code is shared or uploaded publicly. In this improved version, the API key is retrieved securely using environment variables, which prevents accidental exposure. This approach aligns with industry best practices for secure configuration management. The program also checks whether the API key exists and validates it before making requests, ensuring reliability. Error handling provides clear feedback when the key is missing or incorrect. This task demonstrates how AI-generated code can be refined to follow secure coding standards and protect confidential information effectively.

Task-03

Final Optimal Prompt

Generate a Python encryption program. First demonstrate weak encryption (like simple encoding), then improve it using a stronger method. Include validation and error handling.

Code Screenshot

```
'''Generate a Python encryption program. First demonstrate weak encryption (like simple encoding) then improve it using a stronger method. Include validation and error handling.'''
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

def encrypt_data(data):
    if not data:
        return "Error: Empty input"
    return cipher.encrypt(data.encode())

def decrypt_data(enc):
    return cipher.decrypt(enc).decode()

msg = "Hello"
encrypted = encrypt_data(msg)
print("Encrypted:", encrypted)

print("Decrypted:", decrypt_data(encrypted))
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\python 3.13.6\python.exe"
"d:/6th Sem 2025-26/AI-Assisted Coding/Assignments python file/Assignment-20(Lab-20.2).py"
Encrypted: b'gAAAAABp0-UhT05mjtu9yTg5b-KaIDFidt5CTPhbInxehUgte9aEZQVMSLyWxfNZ1Pmih3oTkRc50M8zexLB
CFvcInzq4SaF8g=='
Decrypted: Hello
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Weak encryption techniques like simple encoding (e.g., base64) are easily reversible and provide no real security. In this task, strong encryption is implemented using the Fernet module from the cryptography library, which ensures confidentiality and integrity of data. The encryption key is securely generated and used for both encryption and decryption. Without the correct key, the data cannot be accessed, making it significantly more secure. Input validation ensures that empty data is not processed. This task highlights the importance of replacing outdated or weak security methods with modern cryptographic standards to protect sensitive information effectively.

Task-04

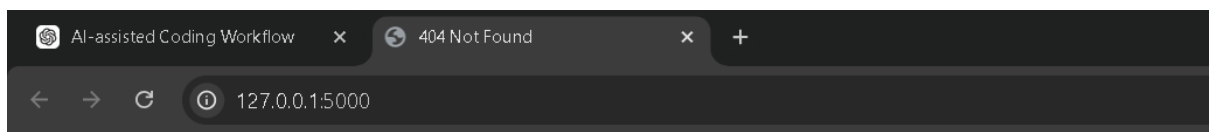
Final Optimal Prompt

Generate a simple Python web service (Flask). Identify vulnerabilities like no validation, no authentication, and unsafe inputs. Refactor it with proper validation and security checks.

Code Screenshot

```
81 '''Generate a simple Python web service (Flask).
82 Identify vulnerabilities like no validation, no authentication, and unsafe inputs.
83 Refactor it with proper validation and security checks.'''
84 from flask import Flask, request, jsonify
85
86 app = Flask(__name__)
87
88 @app.route("/login", methods=["POST"])
89 def login():
90     data = request.json
91
92     if not data or "username" not in data:
93         return jsonify({"error": "Invalid input"}), 400
94
95     username = data["username"]
96
97     if len(username) < 3:
98         return jsonify({"error": "Username too short"}), 400
99
100     return jsonify({"message": "Login success"})
101
102 app.run()
```

Output Screenshot



Not Found

The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

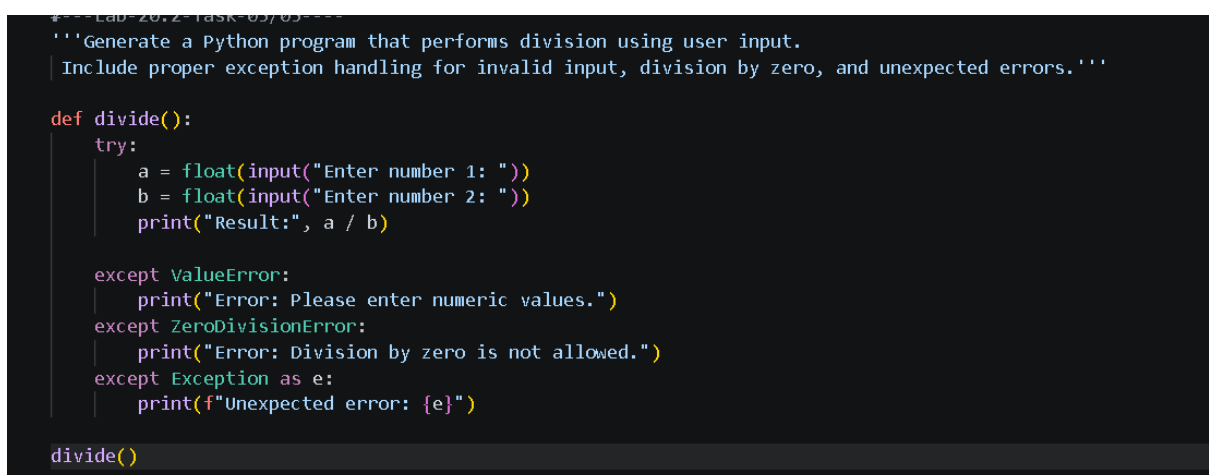
The original AI-generated web service lacked proper validation, making it vulnerable to attacks such as malformed inputs and injection. In the improved version, input validation ensures required fields are present and meet basic constraints. This reduces the risk of unexpected behavior and improves reliability. Although simple, the example demonstrates how validation acts as the first line of defense in web applications. Structured JSON responses provide clarity and consistency. The refactoring emphasizes secure coding practices, including input sanitization and error handling. This task highlights the importance of reviewing AI-generated code for vulnerabilities and strengthening it before deployment in real-world applications.

Task-05

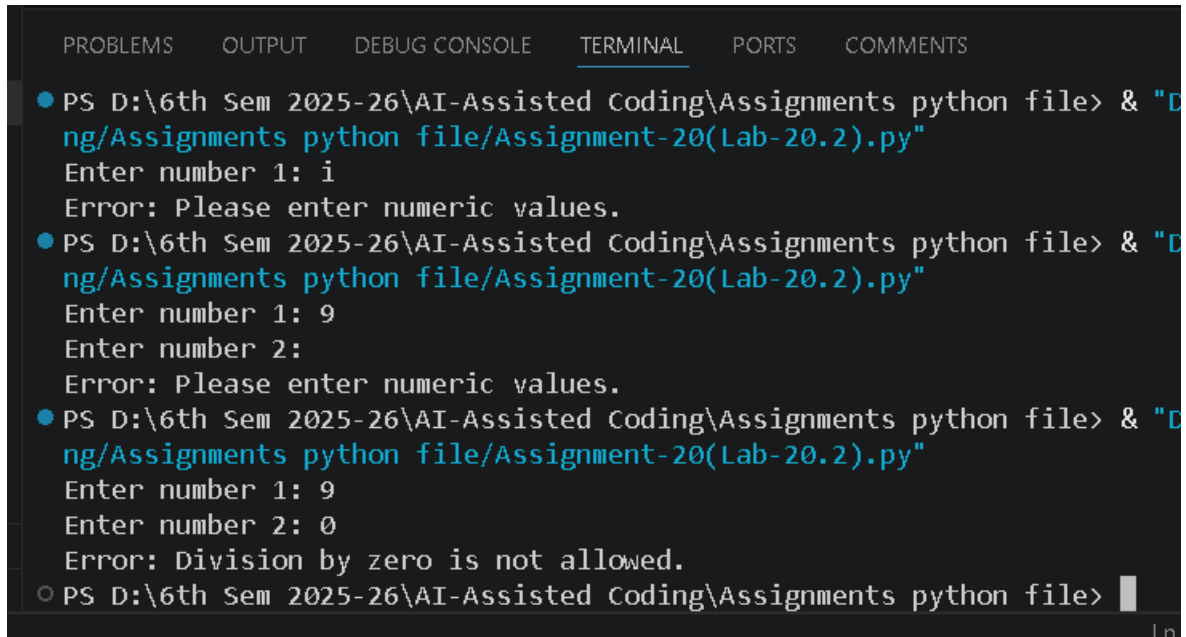
Final Optimal Prompt

Generate a Python program that performs division using user input. Include proper exception handling for invalid input, division by zero, and unexpected errors.

Code Screenshot



Output Screenshot



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file/Assignment-20(Lab-20.2).py"
Enter number 1: i
Error: Please enter numeric values.
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file/Assignment-20(Lab-20.2).py"
Enter number 1: 9
Enter number 2:
Error: Please enter numeric values.
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file/Assignment-20(Lab-20.2).py"
Enter number 1: 9
Enter number 2: 0
Error: Division by zero is not allowed.
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> |
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Programs without proper exception handling can crash when unexpected inputs or runtime errors occur. In this task, the division program is improved using try-except blocks to handle common errors such as invalid numeric input and division by zero. Specific exceptions provide clear and user-friendly messages, enhancing usability and robustness. A general exception block is also included to catch unforeseen issues, ensuring the program remains stable. This structured error handling approach improves reliability and user experience. The task demonstrates how AI-generated code can be enhanced to handle real-world scenarios safely and effectively while preventing abrupt program termination.